

Università degli Studi di Salerno
Corso di Ingegneria del Software

MediBook
ODD
Versione 1.5



Data: 21/12/2025

Progetto: MediBook	Versione: 1.5
Documento: ODD	Data: 21/12/2025

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Christian Casale
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
18/12/2025	1.1	Inizio stesura introduzione e package	Samuele Colucci
18/12/2025	1.2	Stesura e completamento package e class Interface	Samuele Colucci
18/12/2025	1.3	Stesura Componenti off-the-shelf e Design Patterns	Christian Casale
20/12/2025	1.4	Aggiunta diagrammi	Christian Casale
21/12/2025	1.5	Modifiche class interface	Christian Casale
11/01/2026	1.6	Cambio nome package	Lelio Crisci

Contents

1. Introduction	4
1.1. Object design trade-offs	4
1.1.1. Componenti off-the-shelf	5
1.1.2. Design Patterns	5
1.2. Interface documentation guidelines	6
1.3. Definitions, acronyms, and abbreviations	7
1.4. References	7
2. Packages	7
2.1. it.unisa.medibook.model	7
2.2. it.unisa.medibook.modelStorage	8
2.3. it.unisa.medibook.service	9
2.4. it.unisa.medibook.control	9
2.5. Viste (Presentation Layer - View)	10
3. Class interfaces	11
3.1. Package it.unisa.medibook.service	12
4. Glossary	28

Object Design Document

1. Introduction

1.1. Object design trade-offs

Durante la fase di Object Design del sistema MediBook, sono state prese decisioni architetturali che hanno richiesto un bilanciamento tra obiettivi di progettazione contrastanti. Di seguito sono riportati i principali compromessi (trade-offs) adottati:

- **Utilizzo Framework:** Si è scelto di utilizzare framework consolidati come **Spring Boot** e **Hibernate**.
 - *Compromesso:* Sebbene l'uso di framework introduca una maggiore complessità iniziale di configurazione e dipendenze esterne, garantisce una drastica riduzione dei tempi di sviluppo, maggiore sicurezza e aderenza agli standard industriali.
- **Sicurezza vs. Prestazioni (Security vs. Performance):** L'implementazione di filtri di sicurezza rigidi (Spring Security), l'hashing delle password con algoritmo BCrypt e l'obbligo di transazioni ACID sul database introducono un overhead computazionale (latenza aggiuntiva).
 - *Compromesso:* Si è deciso di sacrificare leggermente le prestazioni pure (tempo di risposta) per privilegiare l'**Integrità dei Dati** e la **Conformità GDPR**, ritenute prioritarie in un contesto medico-sanitario.
- **Astrazione vs. Controllo (ORM vs. SQL Puro):** Per la gestione dei dati è stato scelto l'approccio ORM (Object-Relational Mapping) tramite **JPA/Hibernate** anziché l'uso di query SQL native dirette.
 - *Compromesso:* Si accetta una minore ottimizzazione delle query complesse a favore di una maggiore velocità di sviluppo, portabilità del database e protezione automatica contro SQL Injection.

1.1.1. Componenti off-the-shelf

Per ottimizzare i tempi di sviluppo e garantire affidabilità, MediBook integra i seguenti componenti di terze parti:

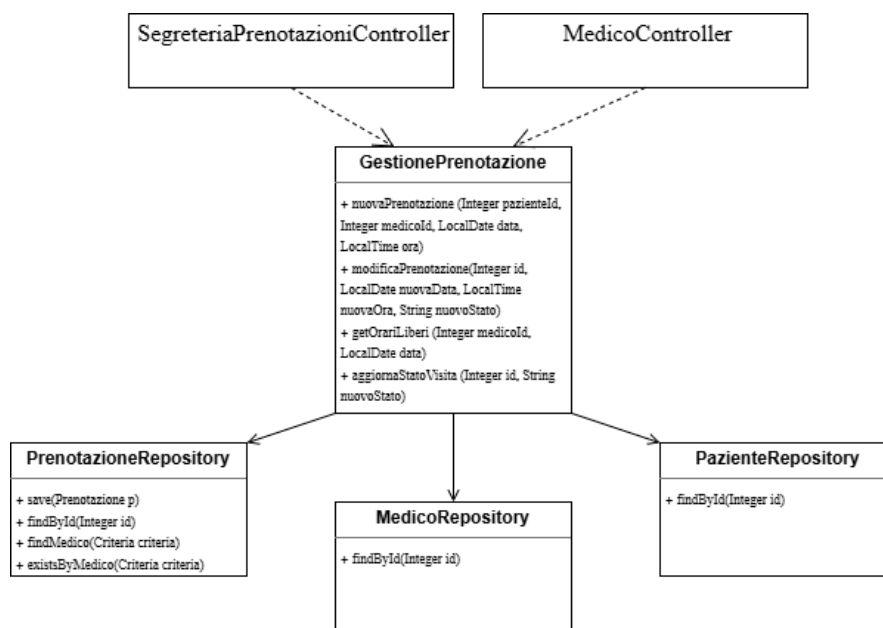
- **Spring Ecosystem (Boot, MVC, Security, Data JPA):** Gestisce l'intera infrastruttura di backend, fornendo configurazione automatica, pattern MVC, sicurezza (autenticazione/RBAC) e astrazione ORM per la persistenza dati.
- **MySQL Connector/J:** Driver JDBC essenziale per abilitare la connessione e le transazioni tra l'applicazione Java e il database MySQL.
- **Jakarta Mail:** Libreria standard utilizzata per l'invio automatizzato di notifiche email (es. conferme prenotazione, recupero credenziali).
- **Frontend Stack (Bootstrap 5 & JSTL):** JSTL gestisce la logica di visualizzazione dinamica all'interno delle pagine JSP.

1.1.2. Design Patterns

- **Gestione Prenotazione (Facade Pattern):**

La classe GestionePrenotazioni agisce come Facade per isolare la complessità della logica di prenotazione dai Controller. Il processo di creazione di una visita richiede l'interazione coordinata con diverse entità (Medico, Paziente) e la verifica di vincoli temporali complessi.

Diagramma:

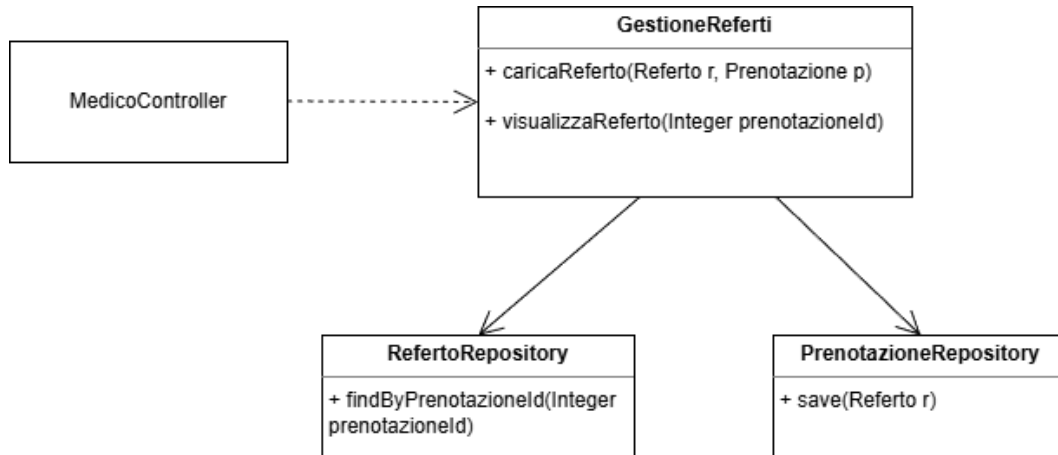


- **Gestione Referti (Facade Pattern):**

La classe GestioneReferti implementa un Facade per garantire l'integrità referenziale

e l'atomicità delle operazioni cliniche. Il metodo caricaReferto() incapsula una transazione critica che coinvolge più tabelle del database.

Diagramma:



1.2. Interface documentation guidelines

Per garantire la manutenibilità, la leggibilità e la coerenza tra i membri del team durante lo sviluppo del sistema con Java Spring Boot e JSP, sono state stabilite le seguenti convenzioni:

- **Naming Classi:** I nomi devono essere sostantivi singolari che rispecchiano l'entità del dominio (es. Prenotazione, Referto, Medico).
- **Naming Metodi:** I nomi devono contenere verbi che descrivono l'azione eseguita (es. `effettuaPrenotazione()`, `caricaReferto()`, `findByData()`).
- **Gestione Errori:** Gli errori di business (es. "Slot orario occupato") non devono essere gestiti tramite codici di ritorno numerici, ma attraverso il lancio di Eccezioni (Java Exceptions), catturate e gestite poi a livello di Controller o Service.
- **Accesso ai Dati (Repository Pattern):** L'accesso ai dati persistenti su MySQL attraverso Interfacce Repository che estendono `JpaRepository` (es. `PrenotazioneRepository`). Questo standardizza le operazioni CRUD e riduce il codice boilerplate.

1.3. Definitions, acronyms, and abbreviations

- ODD: Object Design Document.
- Repository: Pattern architetturale utilizzato per astrarre l'accesso ai dati persistenti
- DTO: Data Transfer Object (Oggetti per il passaggio dati tra i layer)
- API: Application Programming Interface
- MVC: Model-View-Controller
- JPA: Java Persistence API
- OCL: Object Constraint Language
- ORM: Object-Relational Mapping.

1.4. References

- Bernd Bruegge & Allen H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd edition
- Slide del corso di Ingegneria del Software (Prof. De Lucia)
- Documento RAD
- Documento SDD
- Requisiti non funzionali: Sezione 3.3 del RAD

2. Packages

La decomposizione del sistema MediBook riflette un'architettura Web a tre livelli (Three-Tier Architecture) implementata secondo il pattern MVC (Model-View-Controller). Il codice sorgente Java è organizzato per separare nettamente le responsabilità di visualizzazione (Presentation Layer), logica di business (Business Layer) e persistenza dei dati (Persistence Layer).

2.1. **it.unisa.medibook.model**

Questo package contiene le classi Entity (POJO - Plain Old Java Objects) che rappresentano gli oggetti fondamentali del dominio e la struttura dei dati.

Classi Contenute:

- **Utente:** Questa classe rappresenta l'entità base per tutti gli attori del sistema. Definisce gli attributi comuni (come nome, cognome, email, password) e funge da superclasse.
- **Paziente:** Questa classe rappresenta l'attore che usufruisce dei servizi sanitari offerti dalla piattaforma. Estende l'entità **Utente** aggiungendo informazioni anagrafiche specifiche (es. codice fiscale, data di nascita, sesso) e mantiene le relazioni con lo storico delle proprie prenotazioni.
- **Medico:** Questa classe rappresenta il professionista sanitario operante all'interno della struttura. Estende l'entità **Utente** includendo attributi professionali (es. specializzazione, numero albo) e gestisce la relazione con le visite assegnate e i referti emessi.

- **Prenotazione:** Questa classe rappresenta l'evento di una visita medica programmata. Costituisce l'associazione tra un **Paziente** e un **Medico** in uno specifico slot temporale (data e ora), tracciando lo stato del ciclo di vita della prestazione (es. *PRENOTATA*, *EFFETTUATA*, *CANCELLATA*).
- **Referto:** Questa classe rappresenta il documento clinico digitale redatto dal medico al termine di una prestazione. Contiene i dettagli della diagnosi e della terapia prescritta ed è associato in relazione 1-a-1 con una specifica **Prenotazione** conclusa.
- **Recensione:** Questa classe rappresenta il feedback qualitativo lasciato dal paziente dopo una visita. Memorizza la valutazione dell'esperienza e collega l'autore (**Paziente**) al destinatario della prestazione (**Medico**).
- **SegreteriaPrenotazioni:** Questa classe rappresenta l'attore amministrativo con privilegi specifici per la gestione dell'agenda. Estende **Utente** e identifica il personale autorizzato a supervisionare, modificare o cancellare le prenotazioni globali del sistema.
- **SegreteriaUtenti:** Questa classe rappresenta l'attore amministrativo incaricato della gestione delle anagrafiche. Estende **Utente** e identifica il personale responsabile della validazione dei profili medici e della gestione degli account dei pazienti.

2.2. it.unisa.medibook.modelStorage

Rappresenta il Data Access Layer (Persistenza). Contiene le interfacce Repository (basate su Spring Data JPA) che astraggono l'accesso al database.

Interfacce Contenute:

- **PrenotazioneRepository:** Gestisce la persistenza degli appuntamenti e le query per la verifica della disponibilità oraria.
- **MedicoRepository:** Gestisce il recupero dei profili medici e dei relativi turni di lavoro.
- **PazienteRepository:** Gestisce l'accesso ai dati anagrafici dei pazienti per l'associazione alle visite.
- **RefertoRepository:** Gestisce il salvataggio e il recupero dei referti medici associati alle visite concluse.
- **UtenteRepository:** Gestisce le credenziali di accesso e l'autenticazione per tutti i ruoli del sistema.
- **SegreteriaUtentiRepository:** Gestisce la ricerca e la manutenzione amministrativa degli account (medici e pazienti) da parte dello staff.
- **SegreteriaPrenotazioniRepository:** Gestisce la supervisione globale e la modifica manuale

di tutti gli appuntamenti presenti nel calendario del sistema.

- **RecensioneRepository**: Gestisce la memorizzazione e il recupero delle valutazioni lasciate dai pazienti.

2.3. **it.unisa.medibook.service**

Rappresenta il Service Layer (Business Logic). Contiene le classi di servizio che incapsulano la logica applicativa, le regole di validazione e il coordinamento delle transazioni.

Classi Contenute:

- **GestionePrenotazioni**: Il cuore del sistema. Valida la disponibilità degli slot temporali, impedisce sovrapposizioni, gestisce la creazione e la modifica delle visite (nuovaPrenotazione, modificaPrenotazione).
- **GestioneUtenza**: Gestisce la logica di autenticazione (login) e smistamento dei ruoli.
- **GestioneReferti**: Gestisce la logica di caricamento e associazione dei referti, verificando i vincoli di stato della visita (es. la visita deve essere "EFFETTUATA").
- **GestioneMedico**: Gestisce il profilo professionale e la configurazione dei turni lavorativi, dati essenziali per il calcolo delle disponibilità nel calendario.
- **EmailService**: Gestisce l'invio automatizzato di notifiche email.
- **PasswordService**: Gestisce la sicurezza delle credenziali tramite algoritmi di hashing, garantendo che le password vengano cifrate prima del salvataggio nel database.

2.4. **it.unisa.medibook.control**

Rappresenta il Controller Layer (Presentation). Contiene i Controller Spring MVC che gestiscono le richieste HTTP, invocano i Service e selezionano la vista (JSP) da renderizzare.

Classi Contenute:

- **AuthController**: Questa classe rappresenta il componente di gestione della sicurezza e dell'accesso. Gestisce le richieste HTTP relative all'autenticazione (login, logout) e alla registrazione di nuovi utenti, orchestrando la creazione e la distruzione delle sessioni utente.
- **PazienteController**: Questa classe rappresenta il controller dedicato alle funzionalità operative dell'attore Paziente. Gestisce la navigazione nella dashboard personale, la visualizzazione dello storico delle visite e le interazioni per la gestione del profilo e delle

prenotazioni lato utente.

- **MedicoController:** Questa classe rappresenta il controller dedicato alle funzionalità operative dell'attore Medico. Gestisce la visualizzazione dell'agenda lavorativa (calendario), l'accesso ai dettagli dei pazienti e il flusso di compilazione e salvataggio dei referti medici.
- **SegreteriaPrenotazioniController:** Questa classe rappresenta il controller amministrativo focalizzato sulla gestione delle visite. Permette al personale di segreteria di supervisionare l'agenda globale, effettuando operazioni di modifica, cancellazione o conferma delle prenotazioni per conto di medici o pazienti.
- **SegreteriaUtentiController:** Gestisce le richieste per visualizzare la lista degli iscritti, la creazione di nuovi profili paziente con password provvisoria e la manutenzione delle anagrafiche.
- **RicercaController:** Questa classe rappresenta il gestore della navigazione per la funzionalità di ricerca. Intercetta i filtri inseriti dall'utente e restituisce la vista (JSP) contenente l'elenco dei medici che soddisfano i criteri.
- **RicercaRestController:** Questa classe rappresenta l'interfaccia API REST per la ricerca asincrona. A differenza degli altri controller, non restituisce viste HTML ma dati in formato JSON, permettendo l'aggiornamento dinamico dei risultati senza ricaricare l'intera pagina web.

2.5. Viste (Presentation Layer - View)

Il layer di presentazione è costituito da pagine JSP (JavaServer Pages) locate in /webapp/WEB-INF/view/.

File Principali:

- **cambio_password_obbligatorio.jsp:** Modulo di cambio password dopo primo accesso di utente registrato da segreteria.
- **dashboard-agenda.jsp:** Dashboard agenda del della segreteriaPrenotazioni
- **dashboard-gestione-anagrafiche.jsp:** Dashboard modifica dati utente da parte della segreteria utenti.
- **dettagli_medico.jsp:** Pagina di dettaglio accessibile ai pazienti.
- **form_paziente.jsp:** Modulo riutilizzabile per l'inserimento o la modifica dei dati anagrafici di un paziente utilizzato sia dalla segreteria utenti.
- **home.jsp:** Home Page dell'applicazione web.

- **login.jsp:** Modulo di accesso unificato.
- **medico.jsp:** Interfaccia di gestione visite per il medico.
- **medico_scrivi_referto.jsp:** Form di scrittura e salvataggio referto da parte del medico.
- **medicoCalendario.jsp:** Modulo per visualizzazione calendario del medico.
- **paziente.jsp:** Dashboard interattiva per il paziente.
- **paziente_profilo.jsp:** Area riservata a paziente dedicata alla gestione dati da parte del paziente.
- **paziente_visualizza_referto.jsp:** Pagina di lettura referto da parte del paziente.
- **registrazione.jsp:** Modulo di registrazione.
- **risultatiRicerca.jsp:** Pagina visualizzazione lista medici dopo la ricerca.
- **segreteria_scrivi_referto.jsp:** Form di scrittura e salvataggio referto da parte della segreteria utenti.
- **segreteria_utenti.jsp:** Dashboard amministrativa per la gestione degli utenti da parte della segreteria utenti.
- **visualizzaRefertoMedico.jsp:** Pagina di lettura referto da parte del medico.

3. Class interfaces

3.1. Package it.unisa.medibook.service

Questo package contiene i Controller. Qui l'enfasi è sulle Pre-condizioni (cosa deve essere vero prima di chiamare il metodo) e Post-condizioni (cosa è garantito dopo).

GestionePrenotazioni	
modificaPrenotazione	Context: GestionePrenotazioni::modificaPrenotazione(id, nuovaData, nuovaOra, nuovoStato): Prenotazione Pre: ValidazioneInput: -- 1. La prenotazione deve esistere self.prenotazioneRepository.existsById(id) and -- 2. La nuova data non può essere nel passato nuovaData >= LocalDate.now() and -- 3 Lo slot non deve essere occupato da ALTRE prenotazioni not self.prenotazioneRepository.existsByMedicoIdAndDataAndOraAndIdNot(self.prenotazioneRepository.findById(id).medico.id, nuovaData, nuovaOra, id) Post: RisultatoAtteso: -- Il risultato deve essere l'oggetto aggiornato e salvato result.id = id and result.data = nuovaData and result.ora = nuovaOra and result.stato = nuovoStato and self.prenotazioneRepository.findById(id) = result
aggiornaStatoVisita	Context: GestionePrenotazioni::aggiornaStatoVisita(id: Integer, nuovoStato: String): Prenotazione Pre: ValidazioneInput: -- La prenotazione deve esistere per poter cambiare stato self.prenotazioneRepository.existsById(id) Post: StatoAggiornato: -- Il campo stato deve essere quello nuovo passato come parametro result.stato = nuovoStato and -- L'ID deve rimanere invariato result.id = id and -- Data e Ora non devono essere cambiate rispetto a prima (@pre) result.data = self.prenotazioneRepository.findById(id).data and

	<pre> result.ora = self.prenotazioneRepository.findById(id).ora and -- Verifica di persistenza self.prenotazioneRepository.findById(id) = result </pre>
visualizzaVisiteMedico	Context: GestionePrenotazioni::visualizzaVisiteMedico(medicoId: Integer): Sequence(Prenotazione) Pre: MedicoEsistente: -- Il medico di cui cerchiamo le visite deve esistere self.medicoRepository.existsById(medicoId) Post: FiltroCorretto: -- Ogni prenotazione nella lista deve avere l'ID medico uguale a quello richiesto result->forAll(p p.medico.id = medicoId)
visualizzaVisitePaziente	Context: GestionePrenotazioni::visualizzaVisitePaziente(pazienteId: Integer): Sequence(Prenotazione) Pre: IdNonNullo: -- L'ID del paziente deve essere fornito pazienteId <> null Post: RisultatoCoerente: -- 1. Tutti gli elementi della lista restituita devono appartenere a quel paziente result->forAll(p p.paziente.id = pazienteId) and -- 2. Il risultato deve corrispondere esattamente ai dati nel DB result = self.prenotazioneRepository.findByPazienteId(pazienteId)
nuovaPrenotazione	Context: GestionePrenotazioni::nuovaPrenotazione(pazienteId: Integer, medicoId: Integer, data: LocalDate, ora: LocalTime): Prenotazione Pre: ValidazioneInserimento: -- 1. La data non può essere nel passato data >= LocalDate.now() and -- 2. Il medico e il paziente devono esistere nel DB self.medicoRepository.existsById(medicoId) and self.pazienteRepository.existsById(pazienteId) and -- 3. Lo slot orario deve essere libero per quel medico not self.prenotazioneRepository.existsByMedicoIdAndDataAndOra(medicoId, data, ora) Post: NuovaPrenotazioneCreata: -- 1. Verifica che i dati dell'oggetto restituito siano corretti

	<pre> result.data = data and result.ora = ora and result.stato = 'PRENOTATA' and -- 2. Verifica le associazioni (Foreign Keys logiche) result.paziente.id = pazienteId and result.medico.id = medicoId and -- 3. Verifica che il numero totale di prenotazioni nel DB sia aumentato di 1 self.prenotazioneRepository.count() = self.prenotazioneRepository.count()@pre + 1 and -- 4. Verifica di persistenza self.prenotazioneRepository.findById(result.id) = result </pre>
getGiorniLavorativi	Context GestionePrenotazioni::getGiorniLavorativi(medicoId: Integer): Sequence(Integer) Pre MedicoEsistente: -- Il medico deve esistere per poter leggere i suoi turni self.medicoRepository.existsById(medicoId) Post GiorniSettimanaValidi: -- Tutti i numeri restituiti devono essere giorni validi (1=Lun ... 7=Dom) result->forAll(giorno giorno >= 1 and giorno <= 7) and -- Ogni giorno restituito deve essere presente nella stringa 'turni' del medico let turniMedico: String = self.medicoRepository.findById(medicoId).turni in result->forAll(g turniMedico.indexOf(g.toString()) > 0)
visualizzaVisitePerCalendario	Context GestionePrenotazioni::visualizzaVisitePerCalendario(medicoId: Integer): Sequence(Prenotazione) Pre MedicoEsistente: self.medicoRepository.existsById(medicoId) Post FiltroStatoAttivo: -- Ogni prenotazione 'p' restituita deve appartenere al medico E con stato -- 'PRENOTATA' result->forAll(p p.medico.id = medicoId and p.stato = 'PRENOTATA')
getOrariLiberi	Context GestionePrenotazioni::getOrariLiberi(medicoId: Integer, data: LocalDate): Sequence(LocalTime) Pre MedicoEsistente: -- Il medico deve esistere per calcolarne i turni

	self.medicoRepository.existsById(medicoId) Post SlotNonOccupati: -- Nessuno degli orari restituiti (result) deve essere presente nel database delle prenotazioni result->forAll(t not self.prenotazioneRepository.existsByMedicoIdAndDataAndOra(medicoId, data, t)) and -- Poiché OCL non può fare il parsing della stringa "turni" (es. "1:09-13"), -- astraiamo dicendo che ogni orario restituito 't' deve essere compreso -- nella definizione dei turni del medico per quella data. let medico: Medico = self.medicoRepository.findById(medicoId) in result->forAll(t -- (Specifica astratta) L'orario t rientra nella stringa dei turni configurata medico.turniInclude(data.getDayOfWeek(), t))
getVisiteFuture	Context: GestionePrenotazioni::getVisiteFuture(pazienteId: Integer): Sequence(Prenotazione) Pre PazienteEsistente: -- 1. Il pazienteId deve essere fornito pazienteId $\neq$ null and -- 2. Il paziente deve esistere nel database affinché sia possibile recuperarne le visite self.pazienteRepository.existsById(pazienteId) Post SoloVisiteAttive: -- 1. Tutte le prenotazioni restituite appartengono al paziente richiesto result->forAll(p p.paziente.id = pazienteId) and -- 2. La sequenza risultante contiene esclusivamente le visite con stato 'PRENOTATA' -- (Escludendo quindi quelle passate, già refertate o cancellate) result->forAll(p p.stato = 'PRENOTATA') and -- 3. La cardinalità del risultato corrisponde al numero totale di prenotazioni del paziente -- che soddisfano il criterio di stato attivo result->size() = self.prenotazioneRepository.findByPazienteId(pazienteId)->select(p p.stato = 'PRENOTATA')->size()
getVisiteStorico	Context: GestionePrenotazioni::getVisiteStorico(pazienteId: Integer): Sequence(Prenotazione) Pre PazienteEsistente: -- 1. L'identificativo del paziente non deve essere nullo pazienteId $\neq$ null and -- 2. Il paziente deve essere presente nel sistema per poterne consultare l'archivio self.pazienteRepository.existsById(pazienteId) Post ArchivioVisiteConclude:

	-- 1. Tutte le prenotazioni restituite devono appartenere al paziente indicato result->forAll(p p.paziente.id = pazienteId) and -- 2. La sequenza risultante contiene solo visite che hanno terminato il loro ciclo attivo. -- In MediBook, lo storico comprende visite effettuate, concluse (refertate) o annullate. result->forAll(p p.stato = 'EFFETTUATA' or p.stato = 'CONCLUSA' or p.stato = 'ANNULLATA') and -- 3. Definizione formale del set risultante tramite selezione sull'intero insieme delle visite del paziente result = self.prenotazioneRepository.findByPazienteId(pazienteId)->select(p p.stato = 'EFFETTUATA' or p.stato = 'CONCLUSA' or p.stato = 'ANNULLATA')
salvaRecensione	Context: GestionePrenotazioni::salvaRecensione(idPrenotazione: Long, voto: Integer, commento: String, autore: Paziente) Pre VincoliIntegrità: -- 1. La prenotazione deve esistere nel sistema self.prenotazioneRepository.existsById(idPrenotazione) and -- 2. Vincolo di Unicità: Non deve essere già presente una recensione per questa specifica prenotazione self.prenotazioneRepository.findById(idPrenotazione).recensione->isEmpty() and -- 3. Il voto deve rispettare il range della scala di valutazione (es. da 1 a 5) voto >= 1 and voto <= 5 and -- 4. L'autore (paziente) deve essere un utente valido e non nullo autore <> null Post CreazioneFeedback: -- 1. Viene creata una nuova istanza di Recensione associata correttamente alla prenotazione let recensione : Recensione = self.prenotazioneRepository.findById(idPrenotazione).recensione in recensione <> null and -- 2. I dati della recensione devono corrispondere ai parametri in input recensione.voto = voto and recensione.commento = commento and recensione.paziente = autore and -- 3. La recensione deve essere collegata al medico che ha effettuato la prestazione recensione.medico = self.prenotazioneRepository.findById(idPrenotazione).medico
getPrenotazioneById	Context: GestionePrenotazioni::getPrenotazioneById(id: Integer): Prenotazione Pre IdentificativoValido: -- 1. L'ID fornito non deve essere nullo id <> null Post RisultatoRecupero: -- 1. Se la prenotazione esiste nel database, il risultato deve essere l'oggetto corrispondente if self.prenotazioneRepository.existsById(id) then result.id = id and self.prenotazioneRepository.findById(id) = result -- 2. Se la prenotazione non esiste, il risultato deve essere nullo (coerente con

	l'uso di .orElse(null)) else result = null endif
visualizzaVisiteMedico	Context: GestionePrenotazioni::visualizzaVisiteMedico(medicoId: Integer): Sequence(Prenotazione) Pre MedicoEsistente: -- 1. L'identificativo del medico deve essere fornito medicoId $\neq$ null and -- 2. Il medico deve essere presente nell'anagrafica del sistema self.medicoRepository.existsById(medicoId) Post ElencoVisiteAssegnate: -- 1. Tutte le prenotazioni restituite devono essere effettivamente assegnate al medico richiesto result->forAll(p p.medico.id = medicoId) and -- 2. La sequenza risultante corrisponde all'unione di tutte le prenotazioni registrate per quel medico nel database result = self.prenotazioneRepository.findByMedicoId(medicoId) and -- 3. (Vincolo di Integrità) L'ordine della sequenza deve essere coerente con i dati persistenti result->size() = self.prenotazioneRepository.findByMedicoId(medicoId)->size()
ricercaPrenotazioni	Context: GestionePrenotazioni::ricercaPrenotazioni(query: String, filtro: String): Sequence(Prenotazione) Pre ParametriRicevuti: -- 1. I parametri di ricerca possono essere nulli o vuoti, il sistema deve gestire la richiesta in ogni caso true Post RisultatoFiltrato: -- 1. Se il filtro è impostato su 'oggi', il risultato deve contenere solo le prenotazioni della data corrente (filtro = 'oggi' implies result->forAll(p p.data = LocalDate.now())) and -- 2. Se viene fornita una query, il risultato deve contenere solo le prenotazioni i cui pazienti hanno nome o cognome corrispondente (query $\neq$ null and query.trim() $\neq$ "" implies result->forAll(p p.paziente.nome.includes(query) or p.paziente.cognome.includes(query))) and -- 3. In assenza di filtri specifici, il risultato corrisponde all'intera collezione delle prenotazioni (filtro = null and (query = null or query.trim() = "")) implies result = self.prenotazioneRepository.findAll()
inviaNotificheModifica	Context: GestionePrenotazioni::inviaNotificheModifica(id: Integer) Pre EsistenzaDati: -- 1. L'identificativo della prenotazione deve essere fornito e non nullo id $\neq$ null and -- 2. La prenotazione deve essere presente nel database per recuperare i contatti necessari self.prenotazioneRepository.existsById(id)

	Post NotificaMultipla: -- 1. Il sistema deve identificare correttamente i destinatari (paziente e medico) associati alla prenotazione let p: Prenotazione = self.prenotazioneRepository.findById(id) in p.paziente <> null and p.medico <> null and -- 2. Deve essere attivata la procedura di invio email verso il paziente utilizzando i parametri aggiornati di data e ora self.emailService.inviaEmailModifica(p.paziente.email,messaggio, p.paziente.nome, p.medico.cognome, p.data.toString(), p.ora.toString()) and -- 3. Deve essere attivata la procedura di invio email verso il medico per l'aggiornamento dell'agenda professionale self.emailService.inviaEmailModificaMedico(p.medico.email, p.paziente.nome, p.data.toString(), p.ora.toString())
--	--

GestioneUtenza	
registraPaziente	Context: GestioneUtenza::registraPaziente(p: Paziente, passwordInChiaro: String) Pre VincoliUnicità: -- 1. L'indirizzo email non deve essere già presente nel database degli utenti not self.utenteRepository.findByEmail(p.email).isPresent() and -- 2. Il Codice Fiscale deve essere univoco e non associato ad altri pazienti not self.pazienteRepository.existsByCodiceFiscale(p.codiceFiscale) and -- 3. La password fornita in input per la registrazione non deve essere nulla o vuota passwordInChiaro <> null and passwordInChiaro.trim() <> " Post CreazioneAccountSicuro: -- 1. Al nuovo account deve essere assegnato il ruolo predefinito per l'accesso dei cittadini p.ruolo = 'PAZIENTE' and -- 2. La password memorizzata deve corrispondere alla versione crittografata tramite algoritmo di hashing p.password = self.passwordService.hash(passwordInChiaro) and -- 3. L'oggetto Paziente deve essere correttamente persistito nel sistema con un identificativo univoco generato self.pazienteRepository.existsById(p.id)
login	Context: GestioneUtenza::login(email: String, password: String): Utente Pre ValidazioneInput: -- 1. L'indirizzo email deve essere fornito in un formato non nullo email <> null and

	-- 2. La password deve essere presente per procedere alla verifica di sicurezza password $\diamond$ null Post Autenticazione: -- 1. Se esiste un utente con l'email fornita e la password corrisponde al suo hash nel database, viene restituito l'oggetto Utente corrispondente let u: Utente = self.utenteRepository.findByEmail(email) in (u $\diamond$ null and self.passwordService.check(password, u.password)) implies result = u and -- 2. In caso di credenziali errate o utente inesistente, il sistema deve restituire un valore nullo per negare l'accesso (self.utenteRepository.findByEmail(email) = null or not self.passwordService.check(password, self.utenteRepository.findByEmail(email).password)) implies result = null
aggiornaProfiloPaziente	Context: GestioneUtenza::aggiornaProfiloPaziente(idUtente: Integer, telefono: String, indirizzo: String, nuovaPassword: String): Paziente Pre EsistenzaProfilo: -- 1. L'identificativo dell'utente deve corrispondere a un paziente registrato nel sistema self.pazienteRepository.existsById(idUtente) and -- 2. I dati di contatto (telefono e indirizzo) devono essere forniti in formato valido telefono $\diamond$ null and indirizzo $\diamond$ null Post ProfiloAggiornato: -- 1. Le informazioni anagrafiche del paziente nel database devono riflettere i nuovi valori forniti let p: Paziente = self.pazienteRepository.findById(idUtente) in p.telefono = telefono and p.indirizzo = indirizzo and -- 2. Se viene fornita una nuova password, il sistema deve memorizzarne l'hash aggiornato (nuovaPassword $\diamond$ null and nuovaPassword.trim() $\diamond$ ") implies self.pazienteRepository.findById(idUtente).password = self.passwordService.hash(nuovaPassword) and -- 3. Il sistema restituisce l'oggetto Paziente aggiornato per confermare la corretta persistenza result = self.pazienteRepository.findById(idUtente)
getPazienteById	Context: GestioneUtenza::getPazienteById(id: Integer): Paziente Pre IdentificativoFornito: -- 1. L'identificativo del paziente deve essere fornito e non nullo id $\diamond$ null Post RecuperoAnagrafica: -- 1. Se il paziente esiste nel database, il risultato deve corrispondere all'oggetto salvato self.pazienteRepository.existsById(id) implies result = self.pazienteRepository.findById(id) and -- 2. Se non esiste alcuna corrispondenza per l'ID fornito, il sistema deve restituire un valore nullo not self.pazienteRepository.existsById(id) implies result = null and -- 3. Il risultato, se presente, deve mantenere l'integrità dei dati anagrafici associati all'identificativo result $\diamond$ null implies result.id = id

dammiTuttiIPazienti	Context: GestioneUtenza::dammiTuttiIPazienti(): Sequence(Paziente) Pre ParametriRicerca: -- 1. Il metodo non richiede parametri in ingresso, la richiesta è sempre valida true Post ElencoCompletoPazienti: -- 1. La sequenza restituita deve contenere tutti i record presenti nell'anagrafica pazienti del database result = self.pazienteRepository.findAll() and -- 2. (Vincolo di Integrità) Ogni elemento nella sequenza deve avere il ruolo specifico di paziente result->forall(p p.ruolo = 'PAZIENTE') and -- 3. La dimensione della sequenza deve corrispondere al conteggio totale dei record presenti nel repository result->size() = self.pazienteRepository.count()
salvaOAggiornaPaziente	Context: GestioneUtenza::salvaOAggiornaPaziente(p: Paziente) Pre ValidazioneDati: -- 1. L'oggetto paziente fornito non deve essere nullo p <> null and -- 2. Per le nuove registrazioni, l'email deve essere univoca nel sistema p.id = null implies not self.utenteRepository.findByEmail(p.email).isPresent() Post PersistenzaEInizializzazione: -- 1. Se il paziente è nuovo (id nullo), il sistema deve assegnare il ruolo 'PAZIENTE', una password provvisoria e inviare l'email di benvenuto p.id@pre = null implies (p.ruolo = 'PAZIENTE' and p.password = self.passwordService.hash('Medibook123') and self.emailService.inviaEmailBenvenuto(p.email, p.nome, p.cognome, 'Medibook123')) and -- 2. Se il paziente esiste già, i dati anagrafici nel database (nome, cognome, CF, telefono, indirizzo) devono essere sincronizzati con i nuovi valori forniti p.id@pre <> null implies (let esistente = self.pazienteRepository.findById(p.id) in esistente.nome = p.nome and esistente.codiceFiscale = p.codiceFiscale and esistente.indirizzo = p.indirizzo) and -- 3. L'operazione deve concludersi con il salvataggio dello stato aggiornato nel repository self.pazienteRepository.existsById(p.id)
eliminaPaziente	Context: GestioneUtenza::eliminaPaziente(id: Integer) Pre EsistenzaAnagrafica: -- 1. L'identificativo fornito deve corrispondere a un paziente effettivamente registrato self.pazienteRepository.existsById(id) and -- 2. L'ID non deve essere nullo per consentire la ricerca nel repository id <> null Post RimozioneSicura: -- 1. Tutte le prenotazioni associate al paziente devono essere rimosse dal

	sistema prima della cancellazione dell'utente self.prenotazioneRepository.findByPazienteId(id)->isEmpty() and -- 2. Il record del paziente deve essere definitivamente eliminato dal database not self.pazienteRepository.existsById(id) and -- 3. (Integrità Referenziale) Non devono sussistere residui dell'account nelle tabelle collegate dell'utenza not self.utenteRepository.existsById(id)
--	--

GestioneReferti	
visualizzaReferto	Context: GestioneReferti::visualizzaReferto(prenotazioneId: Integer): Referto Pre: IdValido: -- L'ID passato non deve essere nullo prenotazioneId <> null and -- La prenotazione a cui cerchiamo il referto deve esistere self.prenotazioneRepository.existsById(prenotazioneId) Post: CorrispondenzaDati: -- SE il risultato non è nullo, ALLORA deve appartenere alla -- prenotazione richiesta result <> null implies result.prenotazione.id = prenotazioneId and -- Il risultato deve corrispondere esattamente a quello memorizzato -- nel database result = self.refertoRepository.findByPrenotazioneId(prenotazioneId)
salvaNuovoReferto	Context: GestioneReferti::salvaNuovoReferto(prenotazioneId: Integer, contenuto: String) Pre VincoliCaricamento: -- 1. La prenotazione deve esistere nel sistema self.prenotazioneRepository.existsById(prenotazioneId) and -- 2. La visita deve essere in stato 'EFFETTUATA' per poter procedere al referto self.prenotazioneRepository.findByPrenotazioneId(prenotazioneId).stato = 'EFFETTUATA' and -- 3. Il contenuto del referto non deve essere nullo o vuoto contenuto <> null and contenuto.trim() <> "" Post FinalizzazioneVisita: -- 1. Deve essere creata una nuova istanza di Referto associata alla prenotazione con la data corrente let r: Referto = self.refertoRepository.findByPrenotazioneId(prenotazioneId) in r <> null and r.contenuto = contenuto and r.dataCaricamento = LocalDateTime.now() and

	-- 2. Lo stato della prenotazione deve cambiare in 'CONCLUSA' per indicare il termine del processo medico self.prenotazioneRepository.findById(prenotazioneId).stato = 'CONCLUSA' and -- 3. Il referto deve essere correttamente persistito e collegato in modo biunivoco alla visita self.refertoRepository.existsByPrenotazioneId(prenotazioneId)
--	---

GestioneMedico	
dammiTuttiIMedici	Context: GestioneMedico::dammiTuttiIMedici(): Sequence(Medico) Pre: true -- Nessun vincolo specifico per l'invocazione Post: ListaCompleta: -- 1. Il risultato deve corrispondere esattamente al contenuto del repository result = self.medicoRepository.findAll() and -- 2. La dimensione della lista restituita deve coincidere con il numero totale di record nel DB result->size() = self.medicoRepository.count()
getMedicoById	Context: GestioneMedico::getMedicoById(id: Integer): Medico Pre: IdFornito: -- L'ID passato come parametro non deve essere nullo id <> null Post: RisultatoCoerente: -- Se il medico esiste nel database, il risultato non deve essere nullo self.medicoRepository.existsById(id.toLong()) implies result <> null and -- Se il risultato non è nullo, l'ID dell'oggetto restituito deve -- corrispondere all'ID richiesto result <> null implies result.id = id.toLong() and -- Il risultato deve corrispondere esattamente a quanto restituito -- dalla repository result = self.medicoRepository.findById(id.toLong())
ricercaAvanzata	Context: GestioneMedico::ricercaAvanzata(query: String): Sequence(Medico) Pre: PuliziaInput: -- 1. La stringa di ricerca non deve essere nulla per permettere la manipolazione del testo query <> null Post: RisultatoGlobale:

	-- 1. Il sistema deve ignorare i prefissi onorifici (Dr., Dott.) per la ricerca nel database let queryPulita = query.replaceAll("(?i)^dr\\.?\\s+ ^dott\\.?\\s+", " ") in result = self.medicoRepository.cercaGlobale(queryPulita) and -- 2. La sequenza risultante contiene medici la cui anagrafica o specializzazione combacia con la query result->forAll(m m.nome.includes(query) or m.cognome.includes(query) or m.specializzazione.includes(query))
getRecensioniPerMedico	Context: GestioneMedico::getRecensioniPerMedico(medicoId: Integer): Sequence(Recensione) Pre MedicoIdentificato: -- 1. L'identificativo del medico deve essere fornito e non nullo medicoId <> null and -- 2. Il medico deve essere presente nel sistema per poterne filtrare i feedback self.medicoRepository.existsById(medicoId) Post ElencoFeedback: -- 1. Tutte le recensioni restituite devono essere associate esclusivamente al medico richiesto result->forAll(r r.medico.id = medicoId) and -- 2. La sequenza risultante corrisponde all'insieme di tutte le valutazioni persistite per quel professionista nel database result = self.recensioneRepository.findByMedicoId(medicoId) and -- 3. (Integrità Dati) Ogni recensione nella lista deve contenere il riferimento al paziente autore e al voto assegnato result->forAll(r r.paziente <> null and r.voto <> null)
calcolaMediaVoti	Context: GestioneMedico::calcolaMediaVoti(recensioni: Sequence(Recensione)): Real Pre DatiCalcolo: -- 1. La collezione di recensioni passata come argomento non deve essere nulla recensioni <> null Post RisultatoMedia: -- 1. Se la sequenza delle recensioni è vuota, il risultato della media deve essere fissato a zero recensioni->isEmpty() implies result = 0.0 and -- 2. Se sono presenti recensioni, il risultato deve corrispondere alla media aritmetica dei voti contenuti recensioni->notEmpty() implies result = recensioni.voto->sum() / recensioni->size() and -- 3. (Vincolo di Precisione) Il valore restituito deve essere un numero reale compreso nel range di valutazione del sistema result >= 0.0 and result <= 5.0
getSuggerimentiRicerca	Context: GestioneMedico::getSuggerimentiRicerca(q: String): Sequence(String) Pre InputValido:

	-- 1. La stringa di ricerca non deve essere nulla $q \neq \text{null}$ and -- 2. La ricerca deve avere una lunghezza minima di 2 caratteri per ottimizzare le prestazioni del database $q.size() \geq 2$ Post GenerazioneSuggerimenti: -- 1. Se l'input è troppo breve, il sistema deve restituire una sequenza vuota $q.size() < 2 \implies result \rightarrow isEmpty()$ and -- 2. Il risultato deve contenere i nomi dei medici (formattati con il prefisso 'Dr.') e le specializzazioni che corrispondono alla query $result \rightarrow \text{forAll}(s \mid \text{self.medicoRepository.cercaGlobale}(q) \rightarrow \text{exists}(m \mid s = \text{'Dr.'} \cdot \text{concat}(m.nome) \cdot \text{concat(' ')} \cdot \text{concat}(m.cognome) \text{ or } s = m.specializzazione))$ and -- 3. (Vincolo di Ottimizzazione) La lista restituita non deve contenere duplicati e deve essere limitata a un massimo di 6 suggerimenti per non sovraccaricare l'interfaccia utente $result \rightarrow \text{isUnique}(s)$ and $result \rightarrow \text{size}() \leq 6$
--	---

EmailService	
inviaEmailModifica	Context EmailService::inviaEmailModifica(destinatario: String, oggetto: String, nomePaziente: String, nomeMedico: String, nuovaData: String,

	nuovaOra: String) Pre InputValidi: -- 1. L'indirizzo email del destinatario è obbligatorio e non vuoto destinatario <> null and destinatario.size() > 0 and -- 2. I dati fondamentali della modifica non devono essere nulli nuovaData <> null and nuovaOra <> null Post EsecuzioneSenzaErrori: -- Poiché il metodo gestisce internamente le eccezioni (try-catch), -- la post-condizione garantisce che il flusso di controllo ritorni al -- chiamante -- senza interrompere l'esecuzione del programma. true -- (Opzionale: Specifica astratta dell'invocazione) -- Implica che la libreria mailSender sia stata chiamata -- self.mailSender.sentMessages->includes(message)
inviaEmailConferma	Context: EmailService::inviaEmailConferma(destinatario: String, nomePaziente: String, nomeMedico: String, data: String, ora: String) Pre: DatiEmailValidi: -- Il destinatario deve esistere e non essere vuoto destinatario <> null and destinatario.size() > 0 and -- I parametri per compilare il template HTML non devono essere nulli nomePaziente <> null and nomeMedico <> null and data <> null and ora <> null Post: RichiestaInviata: -- Nota: OCL non può verificare se l'SMTP ha consegnato realmente l'email. -- Si assume che il comando 'mailSender.send()' sia stato invocato correttamente. true
inviaEmailBenvenuto	Context EmailService::inviaEmailBenvenuto(destinatario: String, nomeUtente: String) Pre DatiContattoValidi: -- L'indirizzo email è obbligatorio per l'invio destinatario <> null and destinatario.size() > 0 and -- Il nome è necessario per personalizzare il template (es. "Ciao --Mario")

	nomeUtente <> null Post EsecuzioneCompletata: -- Il metodo garantisce la terminazione del flusso senza interrompere -- il processo di registrazione chiamante, gestendo internamente -- eventuali errori SMTP. true
costruisciHtmlModifica	Context: EmailService::costruisciHtmlModifica(nome: String, medico: String, data: String, ora: String): String Pre: ParametriValidi: -- I parametri non devono essere nulli nome <> null and medico <> null and data <> null and ora <> null Post: ContenutoDinamicoPresente: -- La stringa risultante deve includere tutti i dati variabili passati result.indexOf(nome) >= 0 and result.indexOf(medico) >= 0 and result.indexOf(data) >= 0 and result.indexOf(ora) >= 0 and -- Il risultato deve essere un documento HTML completo result.indexOf('<html>') >= 0 and result.indexOf('</html>') >= 0
inviaEmailModificaMedico	Context: EmailService::inviaEmailModificaMedico(destinatarioMedico: String, nomePaziente: String, nuovaData: String, nuovaOra: String) Pre: ParametriObbligatori: -- L'indirizzo email del medico deve essere presente destinatarioMedico <> null and destinatarioMedico.size() > 0 and -- I dettagli dell'appuntamento per il template HTML non devono essere nulli nomePaziente <> null and nuovaData <> null and nuovaOra <> null Post: NotificaGestita: -- Il metodo completa l'esecuzione. -- L'eventuale fallimento dell'invio (SMTP) viene catturato e loggato, -- quindi non influisce sullo stato degli oggetti del dominio. true

PasswordService	
hash	Context PasswordService::hash(passwordInChiaro: String): String

	Pre InputValido: -- La password da cifrare non deve essere nulla o vuota passwordInChiaro <> null and passwordInChiaro.size() > 0 Post SicurezzaGarantita: -- 1. Il metodo deve restituire una stringa valida result <> null and -- 2. L'hash generato deve essere DIVERSO dalla password originale. -- Questo garantisce che non stiamo salvando testo in chiaro nel -- DB. result <> passwordInChiaro
check	Context: PasswordService::check(passwordInserita: String, passwordNelDatabase: String): Boolean Pre: InputValidi: -- Entrambe le stringhe devono essere fornite per il confronto passwordInserita <> null and passwordNelDatabase <> null Post: RisultatoConvalida: -- Il risultato è vero se e solo se la password fornita corrisponde -- all'hash nel DB result = (self.passwordEncoder.matches(passwordInserita, passwordNelDatabase)) and passwordInserita = passwordInserita@pre and passwordNelDatabase = passwordNelDatabase@pre

4. Glossary

Questa sezione fornisce le definizioni dei termini tecnici, degli acronimi e delle tecnologie menzionate all'interno del presente documento di Object Design.

- **BCrypt**: Algoritmo di hashing forte utilizzato all'interno di PasswordService per la cifratura delle password prima della persistenza nel database.
- **CSS (Cascading Style Sheets)**: Linguaggio utilizzato per definire la formattazione e lo stile grafico dei componenti dell'interfaccia utente.
- **DBMS (Database Management System)**: Software progettato per la gestione dei dati; MediBook utilizza **MySQL 8.0**.
- **Invariante (Invariant)**: Vincolo OCL che deve risultare sempre valido per tutte le istanze di una determinata classe.
- **Java**: Linguaggio di programmazione orientato agli oggetti utilizzato per l'intero backend e la logica di business.
- **JavaBeans (Entity)**: Classi che incapsulano i dati del dominio (es. Prenotazione, Paziente), mappate sul database tramite annotazioni JPA.
- **JSON (JavaScript Object Notation)**: Formato per lo scambio di dati utilizzato dalle API REST per aggiornamenti asincroni della UI.
- **JSP (JavaServer Pages)**: Tecnologia utilizzata dai Controller per generare dinamicamente le pagine HTML lato server.
- **MVC (Model-View-Controller)**: Pattern architetturale che separa la gestione dei dati (Model), l'interfaccia utente (View) e la logica di controllo (Controller).
- **OCL (Object Constraint Language)**: Linguaggio formale che permette di specificare vincoli su singoli elementi del modello o gruppi di essi. È un linguaggio non procedurale e non denota il flusso di controllo.
- **Post-condizione (Postcondition)**: Vincolo OCL che definisce lo stato del sistema o il valore di ritorno (result) dopo l'esecuzione di un metodo.
- **Pre-condizione (Precondition)**: Vincolo OCL che deve essere soddisfatto affinché un metodo possa essere eseguito correttamente.
- **Repository (Spring Data JPA)**: Interfacce che gestiscono automaticamente l'accesso ai dati e le query al database.

- **Service Layer:** Livello architetturale che incapsula la Business Logic (es. GestionePrenotazioni), coordinando le transazioni e applicando le regole definite nei contratti OCL.
- **Spring Boot:** Framework utilizzato per semplificare la configurazione e il deployment dell'applicazione Java.
- **SQL (Structured Query Language):** Linguaggio standard per l'interrogazione dei database relazionali.